GridPact: agent usability and final project slides

Homework 9, MAS.665 AI Studio, Fall 2026. Team: Nicolas Hong, Maheen Rassell.

Links	
Interface documentation	https://github.com/nh13120/gridpact/blob/hw9-agent-usability/docs/AGENT_GUIDE.md
Our slides in the shared Demo Day deck	[paste the link to our slides in the shared Demo Day deck here]
Code (GridPact)	github.com/nh13120/gridpact
HW8 pull request (Nanda Town)	github.com/projnanda/nandatown/pull/345
Cold-start plan, logs, scorer	`coldstart/` in the repository: `PLAN.md` (committed before the runs), `runs/run1..3/` (proxy logs, database, scores)

Contributions

Person	Contribution
Nicolas Hong	GridPact itself: the protocol, ledger, MCP server and Haiku agents; the HW7 preregistration, benchmark harness, runs and the v2 change that this work builds on.
Maheen Rassell	HW6 audit fixes (bounty accounting, money-conservation check, settlement tests); HW7 write-up; HW8 Nanda Town integration and pull request; HW9 agent guide, cold-start test bed and scorer, the get_commitments improvement and tests, final slides and this report. The HW8 and HW9 engineering was done with an AI coding assistant (Claude Code), which also ran the cold-start test bed described below.

1. The interface

GridPact is shared infrastructure for energy agents: a marketplace and referee. Agents trade energy for windows of 15-minute steps, a seller locks a deposit (stake) against its promise, plain code settles delivery, and an auditor can challenge a false capacity claim. It is not an agent itself. The full guide, written for an agent that has seen nothing else, is docs/AGENT_GUIDE.md. Summary:

Access	MCP over streamable HTTP, e.g. `http://127.0.0.1:8765/mcp`. The guide gives the three-request HTTP handshake (initialize, notifications/initialized, tools/call) and a working curl example, so any client that can POST JSON can use it.
Tools	register_agent, find_counterparties, propose, respond, get_inbox, get_commitments (added in this homework), get_ledger, challenge. Every reply is JSON with `ok` and, on refusal, a `reason`.
Inputs and outputs	Time is 96 steps of 15 minutes (step 68 = 17:00); energy in kWh per step, prices in $/MWh. The guide tabulates every tool's inputs and returned fields.
Permissions and limits	Roles buyer, seller, auditor; a secret token from registration (shown once) is the identity. Caps on kWh per step; the seller's stake is locked at acceptance; only auditors challenge; a valid challenge pays 10% of the stake, a wrong one costs a $50 bond; one valid challenge per commitment. Agents cannot settle: the operator closes the day.
Setup	`GRIDPACT_DATA_DIR=fixtures/synthetic python -m gridpact.server` (made-up registry, no data files or API key needed).
One small example	COM_09 registers, proposes 500 kWh/step to PVB_17 for steps 68 to 79 at $50/MWh with a $60 penalty (request_id 'order-1'); PVB_17 reads get_inbox and accepts; the commitment locks a $360 stake (60 x 500 x 12 / 1000).

2. Cold-start test

Question. Can an agent that has only the guide, a URL and a task use GridPact for a real job? **Setup.** Three fresh Claude sessions ran at the same time as a buyer (COM_09), an honest seller (PVB_17) and an auditor (AUD_1), each with its own logging proxy in front of one GridPact server on the synthetic registry. Each was given the guide, its URL and a one-paragraph task. No call sequence and no expected answer were given; the author passed no messages between them. They coordinated only through GridPact. The auditor was also authorised to read the public registry file, because the nameplate is only there. A seller PVB_37, which declares 5,000 kW for a 2,500 kW battery, was

registered by its owner as part of the setup, with a small script that accepts any offer made to it.

Success defined before the run. `coldstart/PLAN.md` (commit 74055c8, before run 1) fixes the task texts and 15 checks, scored from the database and the proxy logs rather than from what a client says:

#	Check	Run 1	Run 2	Run 3
B1	buyer registered as COM_09	PASS	PASS	PASS
B2	a commitment covers steps 68 to 79	PASS	PASS	PASS
B3	at least 1,500 kWh/step committed	**FAIL**	**FAIL**	PASS
B4	all prices at or below $55	PASS	PASS	PASS
B5	no double-booking, no cap exceeded	PASS	PASS	PASS
B6	PVB_99 request refused, nothing created, reported	PASS	PASS	PASS
B7	buyer's reported totals match the database	**FAIL**	**FAIL**	PASS
S1	PVB_17 registered as seller	PASS	PASS	PASS
S2	every offer to PVB_17 answered	PASS	PASS	PASS
S3	price floor ($45) and cap respected	PASS	PASS	PASS
A1	an auditor registered	PASS	PASS	PASS
A2	valid challenge on PVB_37 before settlement	PASS	PASS	PASS
A3	no wrong challenge (honest plant not accused)	PASS	PASS	PASS
Y1	ledger hash chain intact	PASS	PASS	PASS
Y2	liar's commitment cut to its real cap; buyer not out of pocket	PASS	PASS	PASS
	Checks passed	**13 of 15**	**13 of 15**	**15 of 15**

What the clients did, and how they used the result

In run 1 the buyer registered, listed sellers, proposed 1,250 kWh/step to PVB_37 and 250 to PVB_17 at $50/MWh, polled its inbox until both were accepted (about a minute), and reported 18,000 kWh for $900. The seller polled for the full ten minutes and accepted the one offer that fit. The auditor wrote its own HTTP client from the guide, found PVB_37's declared 5,000 kW in the ledger, compared it with the registry nameplate (2,500 kW), challenged, and was ruled valid ($90 bounty); it left the honest PVB_17 alone. The server cut PVB_37's commitment to 625 kWh/step and released $450 of its stake. After settlement the ledger chain verified and the liar's delivery matched its real battery.

Where the result use failed (run 1). The auditor's challenge cut the buyer's commitment at 16 s. The buyer next polled at 42 s, but `get_inbox` shows only that an offer is 'accepted', not its current volume. It reported 18,000 kWh and $900; the database held 10,500 kWh and $525 (B3 and B7). **Classification: interface limitation.** The buyer followed the guide, nothing was refused, and no tool could have told it. It is not a test-setup problem: the proxy, the server and the liar's script behaved as designed, and the cut is GridPact working as intended.

Other friction the clients reported (all recorded in the fixes below): no stated way to read the registry parquet file (system Python had no parquet reader; the auditor borrowed the repository's virtual-environment interpreter, which we disclose as a deviation from 'documentation only'); the guide did not list the ledger's action names, the inbox's fields, that steps are inclusive, or that a seller must register before it can be proposed to. The shell's 120-second background rule and a missing `timeout` command on macOS were environment issues, not GridPact's.

3. Boundary case, improvement, rerun

Boundary tested. In every run the buyer also asked for PVB_99 (steps 100 to 110). GridPact answered `{"ok": false, "reason": "PVB_99 is not registered"}`, created no offer, and every client reported it accurately (B6). That refusal fires before the step range is looked at, so the out-of-range case was not actually reached live; `tests/test_boundaries.py` covers it directly (steps outside 0 to 95, reversed steps, over-cap volume, negative price, empty request_id, wrong role, wrong token, a second challenge on one commitment): each is refused with a reason and changes no offers, commitments or ledger entries.

Improvement. A new read tool, `get_commitments`, returns each commitment's current volume, the original volume if it was cut (`trimmed_from_kwh`), price, stake and status; auditors see all, buyers and sellers their own. The guide was

extended with the missing details above.

Rerun. Regression test `tests/test_get_commitments.py` (a buyer sees 1,250 become 625 after a valid challenge; parties see only their own). Then the same three-client workflow ran again with fresh clients and the same checks. **Run 2:** 13 of 15 checks. The tool worked, but the buyer called it once at 46 s, before the challenge at 107 s, then polled only `find_counterparties` until 352 s and reported a stale position (18,000 kWh, $990 against 10,500 kWh and $578 in the database). That is client behaviour on an interface that pushes nothing, plus a guide that did not say a position can change until settlement. The guide now says so explicitly (re-read get_commitments right before acting or reporting), recorded in the plan before the next run.

Run 3: 15 of 15 checks. The auditor challenged PVB_37 at 12.6 s. The buyer, polling for about nine minutes, re-read get_commitments at 557 s, found PVB_37's commitment cut to 625 kWh/step (trimmed_from_kwh 1,250), proposed the missing 625 kWh/step to PVB_17, which accepted, re-read again, and reported 18,000 kWh and $990, matching the database exactly (B3 and B7 pass). The buyer's report says the guide's warning to re-read was essential. Caveat: one run per condition. In run 2 the buyer stopped polling at 352 s; in run 3 it polled for 580 s, so the two runs are not matched in length, and we do not claim the fix works every time.

What improved: the information now exists (an agent can see a cut and the original volume), the guide covers the friction the clients listed, and the first run's blind spot has a regression test. **What did not:** a buyer that does not re-read its position still reports a stale one; nothing is pushed to agents; the nameplate is still only in a file that needs a parquet reader. **Unproven:** the clients are fresh Claude sessions started by the author, not another builder's agents, so this is a **self-run** test with no independent client; with one run per condition the outcome depends on timing and we do not claim a rate; only buyer, seller and auditor roles and one lie were tested.

4. Repeat the test

```
# 1. server, logging proxies and the liar's owner script; prints three client URLs
python -m coldstart.run_env --out coldstart/runs/mine
# 2. give each fresh client docs/AGENT_GUIDE.md, its URL and its task from coldstart/PLAN.md
# 3. score the 15 checks (--settle closes the day as the operator)
python -m coldstart.check coldstart/runs/mine --reports reports.json --settle
# unit, boundary and regression tests (37)
pytest
```

Disclosure: self-run; the clients were fresh Claude subagent sessions of the same model family as the author's assistant; manual help during the runs: none; the author wrote the guide, the test bed and the checks; the clients shared a working directory with the repository and were told to read only the guide (their reports list the files they read; the auditor also used the repository's Python interpreter, noted above). In run 2 the seller's polling ended at about 560 of 600 seconds because the test bed was stopped to start run 3; no offer was pending, so no outcome changed.

Final project slides

Export of our slides. Live deck: [paste the link to our slides in the shared Demo Day deck here]. The embedded demo video is checked in the live deck.

GRIDPACT · MAS.665 AI STUDIO · FALL 2026 1 / 4

THE PROBLEM

Who checks a machine's promise?

AI agents will buy and sell grid power for batteries, solar plants and communities. Their claims are unchecked.

THE PROBLEM

An agent says "I can deliver this much" and that number is whatever its owner typed in. When 1 plant in 4 claimed double its real size, the community received only 65% of the energy it ordered. With no checks, lying earned $355 more per run than honesty.

WHY IT IS AGENTIC

Every party is an autonomous agent acting for an owner with its own interests, at the grid's 15-minute rhythm. No human can referee that. GridPact is the referee that agents call over MCP, so rules, not trust, decide who is right.

WHO USES IT

Buyer agents (communities, data centers). Seller agents (solar plus battery plants). Auditor agents that check claims and are paid for true challenges. Any agent that speaks MCP can join from our written guide.

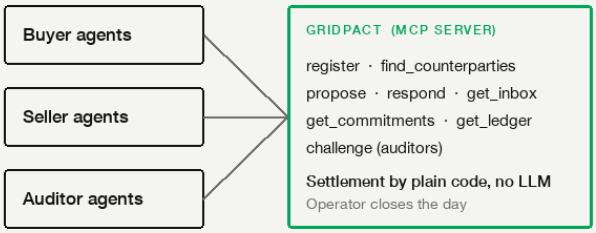
NICOLAS HONG · MAHEEN RASSELL PR GITHUB.COM/PROJNANDA/NANDATOWN/PULL/345

GRIDPACT · MAS.665 AI STUDIO · FALL 2026 2 / 4

WHAT WE BUILT

The referee between agents

Deposit, auditor, deterministic settlement and a tamper-evident ledger behind one MCP interface.



DEPOSIT

A seller locks a stake when it accepts. Missing energy is paid from it to the buyer.

CHECK

An auditor compares the claim with the public spec sheet. True challenge: bounty and the promise is cut to the real size. False: a $50 bond.

WRITTEN FOR AGENTS

docs/AGENT_GUIDE.md: connect, tools, limits, example. A fresh agent used it from nothing.

DEMO VIDEO (EMBED HERE)

2 min: deposit, challenge, settlement

Run it yourself: python demo_deposits.py

NICOLAS HONG · MAHEEN RASSELL PR GITHUB.COM/PROJNANDA/NANDATOWN/PULL/345

WHAT THE EVIDENCE SAYS (HW6 TO HW9)

Measured, then repaired

50/50
liars caught before settlement
275 pre-registered runs, passed twice

90%
of the buyer's need delivered
up from 65% after v2 trims a caught liar

0/160
honest plants wrongly accused
2/160 in version 1

1,520
agents, one day, 6 seconds
scripted load test; found a lock bug, fixed

HW8 NANDA TOWN	Tested through the sandbox, Path mode. Town's duplicate test found a real double-booking bug; we fixed it (retry-safe propose) and 36 of 36 profile runs then passed, with Haiku seller and auditor.
COLLUDING AUDITOR	If the only auditor colludes, the buyer is still refunded by the stake. One independent honest auditor restores the repair (5/5). Framing an honest seller fails and costs the framer $50 each time.
HW9 COLD START	Three fresh agents (buyer, seller, auditor) got only the written guide. Run 1 passed 13 of 15 checks: the buyer could not see that a challenge had cut its promise. After we added get_commitments and a guide note, a fresh buyer found the cut, bought the missing energy and passed 15 of 15 (single runs, self-run).

WORKED, FAILED, NEXT

What we would do next

WHAT WORKED

Deposits plus deterministic settlement make lying lose money. The benchmark was written down before it ran and passed on a full rerun. A fresh agent registered, traded, audited and was refused cleanly when it asked for something unsupported.

WHAT FAILED OR IS UNPROVEN

v2 missed a second liar in 8 of 10 large runs (the stake still charged it). The cold-start buyer could not see its promise had been cut: found, fixed with get_commitments. Delivery is estimated from the registered size, not live meters. The buyer is scripted in the Town runs, and only one lie is tested.

NEXT

A second audit pass after volume is re-sourced. Live meter data and a nameplate tool. Reputation that lowers the stake a plant must post. A shadow-mode pilot with one ENGIE community, no money moving.